Coding Assignment 3: One Array, One Index, Two Jobs

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

September 9, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

3.1 Why this problem

Week 3 gave the calculator two things: a row of cells that finds `A[i]` by arithmetic, and a nesting discipline — `push`, `pop`, `peek` over one array and one index `top` — that checks brackets and evaluates postfix in a single left-to-right pass each. The lecture stated the contract but left the *code* for you to write, with one deliberate simplification you must now unlearn: on the slides, `pop` on an empty stack returns -1 to keep the slide readable. That sentinel cannot tell "the stack held -1" from "the stack was empty" — a real stack must say which happened. This assignment is that real stack, plus the two jobs the lecture promised it could do: Lab P1 (parenthesis check) and Lab P2 (postfix evaluation), both running on your array implementation.

3.2 Your task

Implement the Stack ADT in C as a **fixed array of `int` with an explicit status code on every fallible operation**. The contract is given to you in `stack.h` — *do not modify it.* Copy `starter_stack.c` to `stack.c` and fill in all eight functions:

```
void        stack_init(IntStack *s);
int         stack_is_empty(const IntStack *s);
int         stack_size(const IntStack *s);
StackStatus stack_push(IntStack *s, int x);      /* STACK_OK / STACK_OVERFLOW */
StackStatus stack_pop(IntStack *s, int *out);    /* STACK_OK / STACK_UNDERFLOW */
StackStatus stack_peek(const IntStack *s, int *out);
int         is_balanced(const char *s);          /* P1: 1 if balanced, else 0 */
EvalStatus  eval_postfix(const char *expr, int *out);  /* P2: status + answer */
```

The stack is one array plus `top`. `stack_init` sets `top` to -1 (empty). `push` refuses with `STACK_OVERFLOW` when `top + 1 == STACK_MAX` and changes nothing; `pop` and `peek` report `STACK_UNDERFLOW` on an empty stack and leave `*out` unchanged. `peek` never removes anything. P1 pushes openers as their character codes into the same `int` cells — no second stack needed.

P1 checks nesting, not content. Push every `(`, `[`, `{`; on a closer, return 0 if the stack is empty or the top is not its match, else pop; every other character (letters, digits, spaces, operators)

is ignored, so `(a+b)*[c]` is checked as `()[]`. Balanced iff the stack ends empty. The empty string is balanced; `NULL` returns 0.

P2 evaluates left to right. Integer literals (optional leading `-`, no leading `+`, must fit in `int`) are pushed; on an operator, `b` pops *first* and `a` second, then push apply(op, a, b) — swapping the order breaks `-`, `/`, and `%`. Tokens are separated by spaces or tabs. On *any* non-`OK` status, `*out` is left unchanged. Read the six `EvalStatus` codes in `stack.h` before coding — each error in the table below maps to exactly one of them.

3.3 Constraints

- Representation must be the `IntStack` array in `stack.h` — no linked lists, no heap allocation, no global/static mutable state, no `strtok` shortcuts that hide the token scan.
- Error paths use the explicit status codes — never the lecture's -1 sentinel. A submission whose `pop` returns -1-as-error fails the hidden suite's sentinel-trap tests (pushing the *value* -1 and popping it back must succeed with `STACK_OK`).
- Correctness is graded against the contract; your code must compile with `gcc -Wall -Wextra` without errors or warnings.
- Complexity: `push`, `pop`, `peek` are $O(1)$ worst case each; `is_balanced` and `eval_postfix` each make one left-to-right pass at $O(n)$ time worst case, holding at most `STACK_MAX` integers.

3.4 Worked examples (these are the visible tests)

Example 1 — the stack itself. After `init`: empty, size 0. `push(10)`, `push(20)` both return `STACK_OK`; `peek` gives 20 and size stays 2; `pop` gives 20, then 10; `pop` on the empty stack returns `STACK_UNDERFLOW`.

Example 2 — P1. `is_balanced("")` is 1. `is_balanced("({[]})")` is 1. `is_balanced("({[})")` is 0 (the `}` meets `[`). `is_balanced("(())")` is 0 (an opener never closes).

Example 3 — P2. `eval_postfix("6 2 3 + * 4 -")` returns `EVAL_OK` with answer 26 ($2 + 3 = 5$, $6 \times 5 = 30$, $30 - 4 = 26$). `eval_postfix("20 4 / 3 -")` returns `EVAL_OK` with answer 2. `eval_postfix("5 0 /")` returns `EVAL_DIV_BY_ZERO`. `eval_postfix("2 +")` returns `EVAL_TOO_FEW_OPERANDS`.

3.5 What to submit

1. `stack.c` — your completed implementation (edit `starter_stack.c`).
2. `stack.h` — unchanged, but include it so the submission compiles standalone.

3.6 Getting started

1. Copy `starter_stack.c` to `stack.c` and fill in the eight function bodies.
2. Sanity-check locally:

```
gcc -Wall -Wextra -o test_visible test_visible.c stack.c
./test_visible
```

All three worked examples should pass.

3. Submit `stack.c` and `stack.h`. The autograder compiles your `stack.c` against a *hidden* test suite and reports pass/fail. The hidden suite covers: the -1 sentinel trap; filling the stack to exactly `STACK_MAX` and one past it; `peek` purity and `NULL`-out calls; empty/single-bracket inputs, every bracket type, 100-deep (fits) vs. 101-deep (capacity) nesting, ignored non-bracket characters, and `NULL`; single-number, negative, and multi-digit postfix; division truncation and division-by-zero; too-few-operands and leftover-operand inputs; bad tokens; empty/whitespace input; a 101-operand overflow; and seeded random batches of bracket strings and postfix expressions checked against an oracle.